GitHub Copilot – TypeScript

Website Development With TypeScript and Copilot

Environment:

Since we are going to work with HTML, CSS, JavaScript and TypeScript, there is no setup to begin with. You just need to decide on a folder to create your files.

You do not need GitHub Copilot Professional to get a lot from this bootcamp but it does help to have it installed. There is a free *limited* version. Here are the instructions for that version: https://docs.github.com/en/copilot/managing-copilot/managing-copilot-as-an-individual-subscriber/getting-started-with-copilot-on-your-personal-account/getting-started-with-a-copilot-plan#accessing-copilot-free

Although the environment that I am using is a TypeScript/web development one, the ideas presented can be adapted to any programming environment. In the beginning we will use the Ask mode of Copilot.

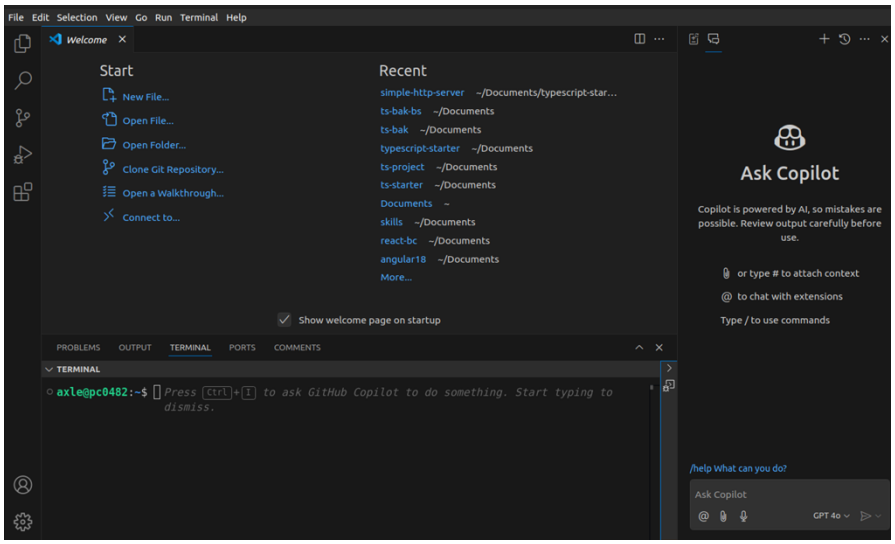
Part 01: Folder Setup

Decide on a folder to start from, e.g. Documents. Open VS Code, you don't have to navigate to a folder yet. Once in VS Code sign in to your GH Copilot account. Sign in to Copilot. Type in your first prompt in the Chat window:

`In the Documents folder/directory create a TypeScript project folder to support a HTML,CSS, JavaScript project.`

Once you have a workspace, you can change the folder name to ghc-ts or something similar.

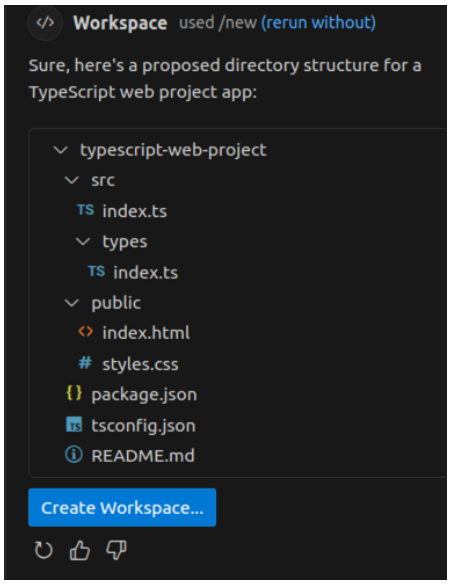
Open a terminal window inside the VS Code IDE. In the menu area there should be an option called *Terminal*, just click it and choose *New Terminal*.



If you see a column with the *DEBUG CONSOLE* label at the top, just click the downward facing arrow and that window will go away.

Depending on your version you may already start seeing Copilot prompts in a dark grey colour in the terminal window. This is called Ghost Text, it is already trying to help you get started.

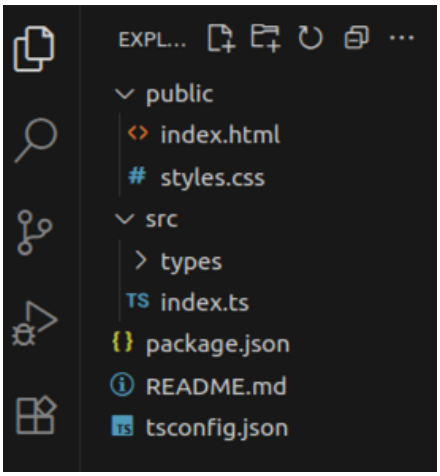
This is how the system responded on my Linux based system. Note I am currently in Ask mode using the model GPT-4.1.



At this point, just follow the suggestions. So in this case I will click the Create Workspace… button. That action will take you the OS and the option to create a project (parent) folder. The tool will most likely supply a name for the project folder, you could always change this from the OS.

It will invite you to open the project or workspace.

This is what my project looks like:

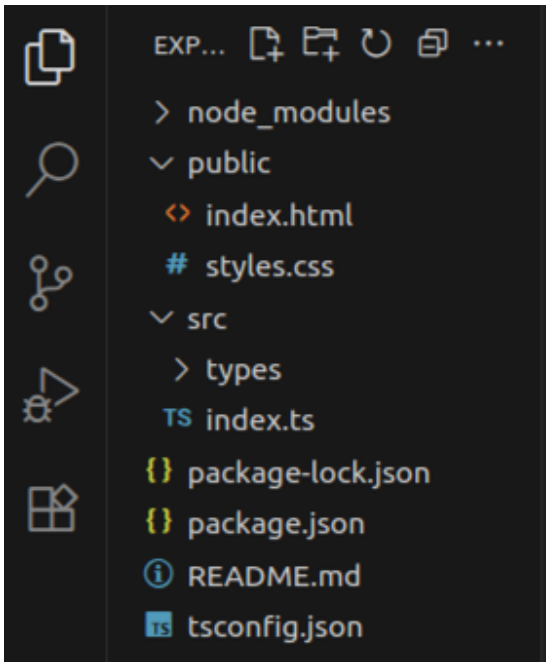


1. If you got this far, check your package.json file. If you have a TS version less than 5, ask the tool to upgrade to the latest TypeScript version, currently 5.8.3
   This is the prompt that I would use:

   `Upgrade this TypeScript project to the latest version of TypeScript`

Part 2

At this point, you should have a TypeScript project folder and all the packages we need for now. If you have everything, run the “npm install” command in the terminal window.



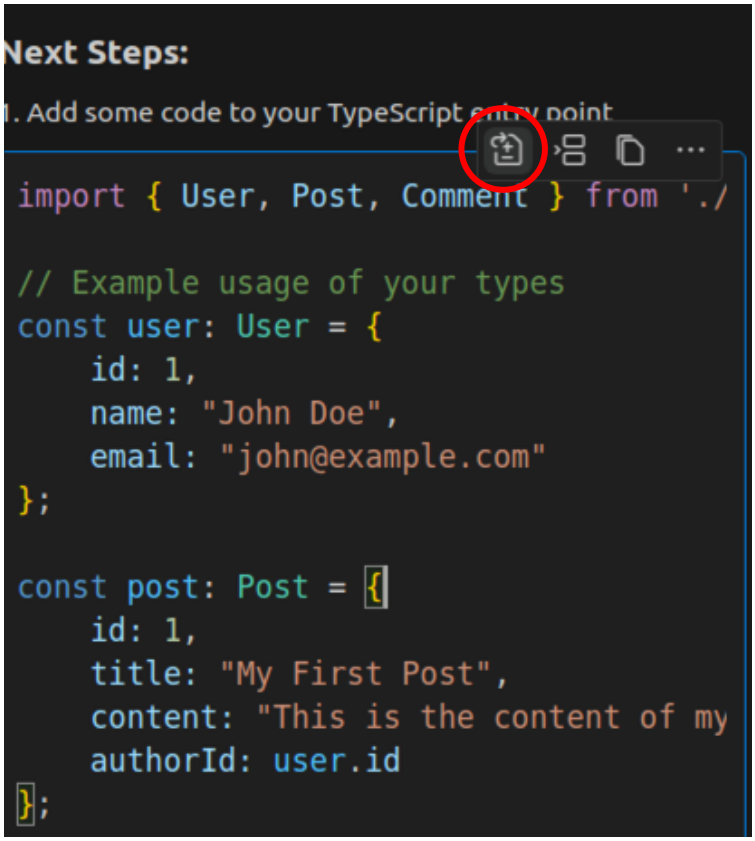
This is what my setup looks like now after running the “npm install” command.

If we were using Copilot to learn TS, we could now ask it how to run the program. So how can we see something on the browser.

Here is my prompt:

Analyze this project and tell me what is the next step, for example how can I see something on the browser.

In my case Copilot gave me some starting code to put into my index.ts file. You may see something lke this:



If you did see something like this, make sure your index.ts file is opened and then click the circled icon. It should have given you several terminal commands. You do not have to execute all of them if you are familiar with this type of development.

If you get a command about build and start then execute those in sequence. If you did this, and if it worked, you should now have a dist folder with a JS file in it. This is the compilation process at work.

In my case the initial setup did give me a server called “lite server”, however if you type npm start as it suggests, you may not see anything on the browser. Now you can just tell Copilot that “the browser opened but it Cannot Get /” and Copilot will walk you through the configuration to set that up. I am going to skip that step and just use VS Code’s Live Server. Just right click on the index.htlm file in the public folder and choose open with Live Server.

Since we would all have slightly different project folders at this point, lets just restart with the zipped file that I have provided for you. It is called ghc_ts_pt02.zip. Just unzip that file and you will have all the files and folders for the next part.

For part3, unzip the part 2 files, open the project folder in VS Code, run “npm install”, then “npm run build”. After that you should have a *dist* folder. Open the *index.html* file in the public folder with Live Server.

If you get an error in the browser, check your paths.

Part 3

If you are using the provided file, lets continue by adding a menu.

Build a menu for my website, I will have four navigable pages, home, products, contact-us and login. For this site, each page will be a physical html page.

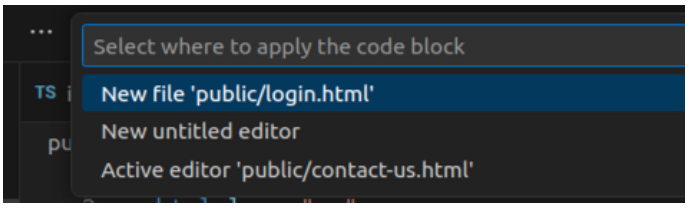
The reason I added the part about physical html page at the end is so that the tool does not try to give me a lot of JavsScript as pointers to blocks of code. I want to work with individual html pages.

Add a theme to the website based on the colors at codecademy.com. Also make the menu horizontal and remove the bullet points.

You can now use the tool to add CSS to make the look and feel a bit nicer.

Create the other three web pages for products, contact-us and login. Use the same theme for all pages.

Note, you will be prompted to create the three pages, be sure you follow the instructions at the top of VS Code and not add this code to the existing index.html file:



At this point, you should have a fully functional menu. It even comes with a hover effect. The project folder at the end of part 3 is called

Part 4

Lets add a form to the contact-us html file.

Add a form to the contact-us.html file, the form should have a username, an email and a text area for adding longer text.

In my case, the AI tool wanted me to create a new .ts file called contactForm.ts in the src folder. The file was created but now, the html file dose not know that the .ts file exists. So all we need to do at this point is tell Copilot to make sure that the html is dependent on the .ts file.

Connect the contactForm.ts file to the contact-us.html file so that the html form is validated by that TypeScript code.

In my case, the tool reacted that I should make sure that I run a build to create the corresponding js file.

Lets do one more prompt before we test.

I want the form to be validated only by TypeScript, so not by the browser's native code. Make sure that the username does not contain numeric values and the email is a valid email.

In my case, this prompt did not work properly. If the code encountered an error in one of the fields, it addes a red border around the box. What I really wanted is for actual text of the error to show up, so here is a follow up prompt.

If the user violated any of the validation I added, create an appropriate error message and add it below the offending input box in red.

Now the form is behaving better but I wanted to add this prompt:

Activate the error message only if the user tabs away from the offending form control, not when the user is actively typing.

Its much better now, but you can take this to further levels. For example a message should not appear if the user has not "touched" the field.

Part 5

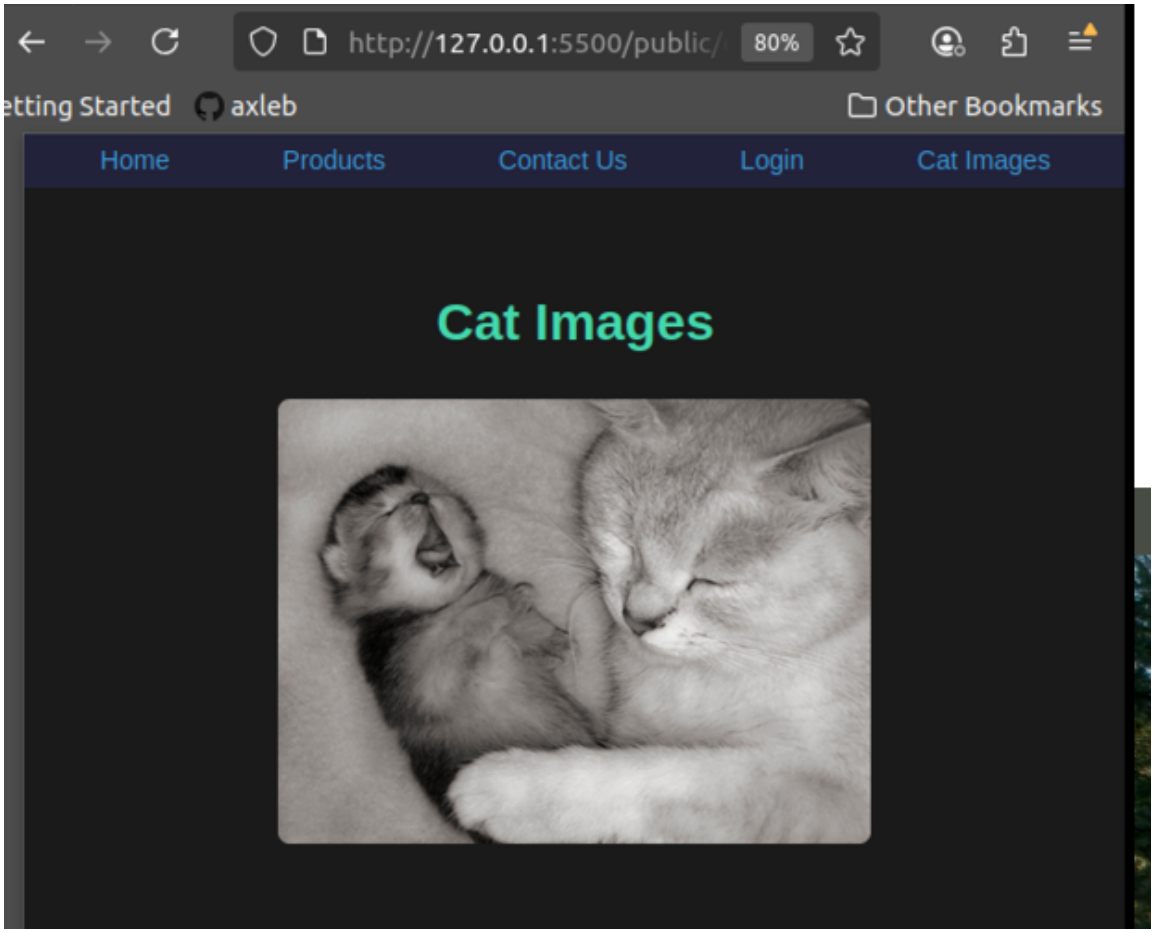
In part 5 we will use the files we have to hit a public API and get some data, a random image to be exact. So far we have four html pages and two TypeScript files to match two of those html pages. Copy and paste this prompt into the prompt input box:

Create a new web page and typescript file called cat_images. Add this web page to the existing menu.

In my case, the AI tool not only created the files, it also created a fetch() function to go to the endpoint api.thecatapi.com/v1/images/search! So now if I go to that web page, it retrieves a random cat image and display the image in the middle of my cat_images.html web page.

It also gave me a line of html code that I needed to add to each existing web page so that this new page can be accessed from the menu.

If you have agent mode in your subscription to GitHub, you will not have to update each page individually, agent mode will update the entire website.



If you went to this public website, you will notice that it is possible to get up to 10 images for free. Lets try to get the page to deliver more images:

`Amend the fetch to retrieve 8 images instead of just one`

hopefully you are seeing 8 images on the web page. Lets try to organize the images a bit and try to get more information if possible

`Create a table to show each image in a row and the breed if any. Add a colourful border around each cell.`

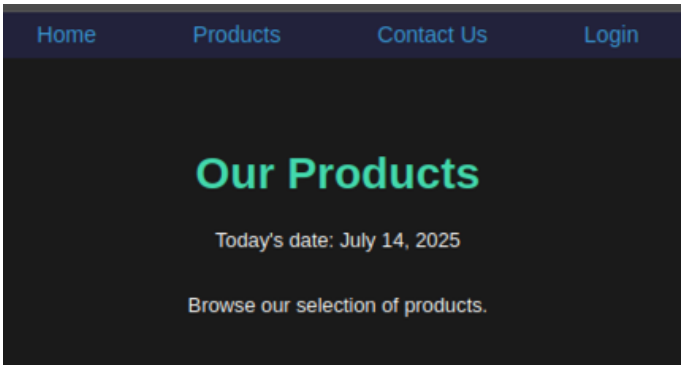
The public website is:
https://developers.thecatapi.com/view-account/ylX4blBYT9FaoVd6OhvR?report=bOoHBz-8t

Part 6 – Dates

In part 5 we will work with dates. Dates in JavaScript/TypeScript are difficult to work with, although recent a more modern Temporal API is being introduced. Currently is it not native but can be used with a polyfill.

1. Here is the first prompt:
   `Use the products page to display the current date just below the page heading. Add a separate typescript file to handle this.`

This is what my page looks like now:



2. To change the format:
   `Change the date format in the products.html file to be mm/dd/yyy`

   In my case, the tool used a function that is not in any library currently installed in my system. It suggested that I change my target in the tsconfig.json file

   Property 'padStart' does not exist on type 'string'. Do you need to change your target library? Try changing the 'lib' compiler option to 'es2017' or later.

   You can fix this error by updating your tsconfig.json to include "es2017" or later in the "lib" compiler option so that padStart is available.

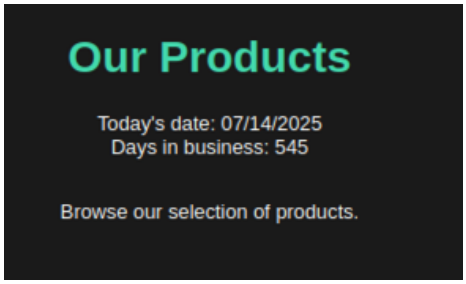
   { "compilerOptions": { // other options "lib": ["dom", "es2017"] // other options } }

   I followed the instructions and updated my tsconfig.json file. I could not get Copilot to do it directly, I had to open the config file then transferred the response to the main chat window, then applied the fix:

```
  "rootDir": "./src",
  "lib": [
    "dom",
    "es2017"
  ]
 },
 "include": ["src/**/*"],
 "exclude": ["node_modules", "**/*.spec.ts"]
```

3. Date arithmetic in JavaScript is difficult but with Copilot and TypeScript it is very easy. Here is my next prompt:

   The company started business on January 16, 2024. below the date, add a second line that shows how many days the company has been in business.



4. We can accomplish any date arithmetic with Copilot, here is a fun line to add:

   Just below the last line you just added, add a new line showing the number of days to Christmas for the current year. Use a bigger font and make it dark yellow in color



Part 7 – Types in TypeScript

The number one reason for using TypeScript is the fact that we can define our own types.

1. Open the index.ts file and pass this prompt in:
   In this index.ts file, add a typical employee type
2. Here is the response:

```
type Employee = {
  id: number;
  name: string;
  email: string;
  department: string;
  position: string;
  salary: number;
  hireDate: Date;
};
```

3. Now as a beginner TS programmer, you might want to know how to use this new Employee type you have. Here is the prompt:
   how can i use this type , show me an example

```
// Create an employee object

const employee1: Employee = {
  id: 1,
  name: "John Doe",
  email: "john.doe@company.com",
  department: "Engineering",
  position: "Software Developer",
  salary: 75000,
  hireDate: new Date("2023-01-15")
};
```

   You can then print this out to the browser or to the console window.

4. What if we wanted to find out about literal types in TS, well just ask:
   Show me an example of literal types and how to use them

```
// String literal types
type Department = "Engineering" | "Marketing" | "Sales" | "HR";
type Position = "Junior Developer" | "Senior Developer" | "Manager" | "Director";

// Number literal types
type ExperienceLevel = 1 | 2 | 3 | 4 | 5;

// Boolean literal types
type IsActive = true; // only true is allowed
```

5. The tool will probably give you an employee called employee1 and when that employee object is created, you will see the use of the literal types above. I have highlighted them here for you:

```
const employee1: Employee = {
  id: 1,
  name: "John Doe",
  email: "john.doe@company.com",
  department: "Engineering", // Must be one of the literal values
  position: "Senior Developer",
  salary: 75000,
  hireDate: new Date("2023-01-15"),
  experienceLevel: 3,
  isActive: true
};
```

In other words, if you made a mistake and typed Engineerings instead of Engineering, you will get an error in the editor.

Part 8 – Higher Order Functions

For this part, I removed all the existing code in the index.ts file and in the Chat window, I right-clicked and choose New Chat.

1. Here is my prompt:
   Use this index.ts file to show me a simple example of how higher order functions work in typescript

```
// Example 1: Function that takes another function as a parameter

function applyOperation(x: number, y: number, operation: (a: number, b: number) => number): number {
        return operation(x, y);
}

// Basic math operations
const add = (a: number, b: number): number => a + b;

// Example 1 usage
console.log("Addition:", applyOperation(5, 3, add)); // 8
```

Note, the tool gave several examples but here I am showing just one, the addition example

2. Lets try to be a bit more concise with our examples. Here is a prompt asking about Optional Chaining in TS:
   show me an example of optional chaining typescript, use a very simple example and no more than 12 lines of code

```
// Simple optional chaining example
interface Person {
  name: string;
  address?: {
    street: string;
    city: string;
  };
}

const person: Person | undefined = { name: "John" };

// Safe access with optional chaining
console.log(person?.name); // "John"
console.log(person?.address?.city); // undefined (no error)
```

3. You can now ask Copilot to explain the code to you. Basically we can create a person class using the Person interface. If we just pass a value for name, we will get a person object, but the address will be missing.
   The idea here is that we can access the person class safely. In the second log() statement, we are trying to print the city. Normally in JS this would cause a runtime error but here it is just undefined.

4. I then asked Copilot this prompt:
   `How can I print "No Data" instead of undefined for line 15`
   Line 15 is that second log() that prints “undefined” in the console window.

   ```
   ...
   console.log(person?.address?.city ?? "No Data" ); // "No Data"
   ```

   Copilot suggested that I use the nullish coalescing operator which is the (??) in the line of code above

Part 9 – Classes

Lets see if Copilot knows how to build a class and explain how it works.

1. Here is my first prompt to the tool to build an example class:

   Use the index.ts file to show me an example of a short class, just 2 properties and 2 methods. The class will be that of an employee. Use the class to show me some of the latest techniques available from typescript

   Here is the suggested code, notice that it has given my TS v 4.9 by default

```
// Using satisfies operator for type safety (TS 4.9+)
type EmployeeRole = 'developer' | 'manager' | 'designer';

// Modern class with decorators and advanced features
class Employee {
 // Private fields with # syntax (ES2022)
 #id: number;

 // Public readonly property
 readonly name: string;

 // Property with satisfies operator for type narrowing
 role: EmployeeRole;

 // Constructor with parameter properties
 constructor(
  id: number,
  name: string,
  role: EmployeeRole
 ) {
  this.#id = id;
  this.name = name;
  this.role = role satisfies EmployeeRole; // Type assertion with satisfies
 }

 // Method with template literal types and const assertions
 getInfo(): `Employee: ${string} (${EmployeeRole})` {
  return `Employee: ${this.name} (${this.role})` as const;
 }

 // Method using optional chaining and nullish coalescing
 promoteEmployee(newRole?: EmployeeRole): Employee {
  const updatedRole = newRole ?? this.role;
  return new Employee(this.#id, this.name, updatedRole);
 }

 // Getter using private field
 get employeeId(): number {
  return this.#id;
 }
}
```

```
// Usage with const assertions and type inference
const emp = new Employee(1, "Alice Johnson", "developer");
console.log(emp.getInfo());
console.log(`ID: ${emp.employeeId}`);

const promoted = emp.promoteEmployee("manager");
console.log(promoted.getInfo());
```

2. I am now going to ask Copilot if it can upgrade to the latest version, which is 5.8.3 currently:
   Can you upgrade this example to TypeScript v. 5.8.3. Answer below:

```
// Using satisfies operator for type safety (TS 4.9+)
type EmployeeRole = 'developer' | 'manager' | 'designer';

// Using 'using' keyword for automatic resource disposal (TS 5.2+)
class DatabaseConnection {
 [Symbol.dispose]() {
  console.log('Database connection closed');
 }
}

// Modern class with decorators and advanced features
class Employee {
 // Private fields with # syntax (ES2022)
 #id: number;

 // Public readonly property with auto-accessor (TS 4.9+)
 readonly accessor name: string;

 // Property with satisfies operator for type narrowing
 role: EmployeeRole;

 // Constructor with parameter properties and using declaration
 constructor(
  id: number,
  name: string,
  role: EmployeeRole
 ) {
  this.#id = id;
  this.name = name;
  this.role = role satisfies EmployeeRole;
 }

 // Method with template literal types and const type parameters (TS 5.0+)
 getInfo<const T extends string = "default">(
  prefix?: T
 ): `${T extends "default" ? "Employee" : T}: ${string} (${EmployeeRole})` {
  const actualPrefix = (prefix ?? "Employee") as T extends "default" ? "Employee" : T;
  return `${actualPrefix}: ${this.name} (${this.role})` as const;
 }
```

```
        // Method using switch(true) pattern with exhaustive checking (TS 5.8+)
         promoteEmployee(newRole?: EmployeeRole): Employee {
          using connection = new DatabaseConnection(); // Auto-disposal (TS 5.2+)

          const updatedRole = newRole ?? this.role;

          // Pattern matching with switch(true) for better control flow
          switch (true) {
           case updatedRole === 'manager' && this.role !== 'manager':
            console.log('Promoting to manager role');
            break;
           case updatedRole === this.role:
            console.log('No role change needed');
            break;
           default:
            console.log(`Changing role from ${this.role} to ${updatedRole}`);
          }
          return new Employee(this.#id, this.name, updatedRole);
         }
         // Getter using private field with improved inference (TS 5.8+)
         get employeeId(): number {
          return this.#id;
         }

         // Static method with const type parameters (TS 5.0+)
         static create<const TRole extends EmployeeRole>(
          id: number,
          name: string,
          role: TRole
         ): Employee & { role: TRole } {
          return new Employee(id, name, role) as Employee & { role: TRole };
         }
        }

        // Usage with const assertions and improved type inference (TS 5.8+)
        const emp = Employee.create(1, "Alice Johnson", "developer" as const);
        console.log(emp.getInfo("Staff"));
        console.log(`ID: ${emp.employeeId}`);

        const promoted = emp.promoteEmployee("manager");
        console.log(promoted.getInfo());

        // Using declaration example (TS 5.2+)
        function processEmployee() {
         using connection = new DatabaseConnection();
         // Connection automatically disposed at end of function
         console.log('Processing employee data...');
        }

processEmployee();
```